A Polynomial-Time Algorithm for Three-Machine Unit-Job Scheduling

OpenAI (with community contributions)

October 27, 2026

Abstract

We give a uniform deterministic polynomial-time algorithm for scheduling unit-length jobs with arbitrary precedence constraints on three identical parallel machines. The algorithm constructs a schedule of minimum makespan and decides exactly whether all jobs can finish by a specified deadline. The proof reorganizes feasible schedules into intervals whose job sets have descriptions of bounded size. A dynamic program searches a family containing polynomially many such descriptions. Global boundary conditions and simplification of inherited information keep the descriptions bounded throughout the decomposition.

1 Introduction

A precedence-constrained schedule must respect two kinds of restrictions: jobs related by precedence must run in order, while unrelated jobs compete for the available machines. Even when every job takes one unit of time, the interaction between these restrictions is subtle. The two-processor case was treated by Fujii, Kasami, and Ninomiya [7, 8], and Coffman and Graham gave an optimal list-scheduling algorithm [2]. Gabow gave an almost-linear-time algorithm for two processors [9], and Garey and Johnson extended the two-processor case to individual deadlines [10] and to release times together with deadlines [11]. Allowing the number of machines to be part of the input yields an NP-complete problem [29, Theorem 1]. For this general problem, Lenstra and Rinnooy Kan showed that deciding whether a schedule of length three exists is already NP-complete, so no polynomial-time algorithm for it has a worst-case ratio below $4/3$ unless P $=$ NP [21]. With the number of machines as the parameter, it is W[2]-hard [1]. The fixed-three decision problem appears as OPEN8 in Appendix A13 of Garey and Johnson [13] and has remained a central complexity question in scheduling [26]. Lenstra and Rinnooy Kan discussed it as one of two open problems in precedence-constrained scheduling [22], its status was reviewed again in Johnson's NP-completeness column [18], and it appears in lists of open problems on approximation [28] and on parameterized complexity [25, Open Problem 6]. The classical results on precedence-constrained scheduling are surveyed by Lawler, Lenstra, Rinnooy Kan, and Shmoys [20].

We consider the following precise decision problem. The input is an explicitly listed directed acyclic graph $G = (V, E)$, with $V = \{1, \dots, n\}$, and an integer T satisfying $1 \le T \le n$. Every vertex is a nonpreemptive job of processing time one. The question is whether there is a function

$$\tau : V \longrightarrow \{1, \dots, T\}$$

such that each value is taken at most three times and $\tau(u) < \tau(v)$ for every $(u, v) \in E$. There are no release dates, communication delays, eligibility restrictions, or additional resources. Slot t means

execution during $[t-1, t)$, and the makespan is the last completion time. Allowing nonnegative real start times would not improve the optimum: fix the machine assignment and within-machine order of a feasible schedule. The graph augmented by machine-order arcs is acyclic, as witnessed by the original schedule. In a topological order of this graph, set each start time to the maximum of zero and its predecessors' completion times. Induction gives integer start times, and no completion time increases. The standard notation for the problem is $P3 \mid \text{prec}, p_j = 1 \mid C_{\max}$ [16].

Theorem 1.1. *Let an explicitly listed finite directed acyclic graph specify the precedence constraints on $n \geq 1$ nonpreemptive unit-length jobs on three identical machines. There is a uniform deterministic algorithm that constructs a feasible schedule of minimum makespan. Given also an integer deadline $1 \leq T \leq n$, it decides feasibility exactly and returns a schedule whenever the answer is affirmative. Both tasks can be performed in $O((L+2)^{150020})$ steps on a deterministic multitape Turing machine, where L is the total binary input length.*

The theorem resolves the polynomial-time side of this three-machine complexity question. Its scope is the explicit model just stated. The polynomial supplied by the proof has a very large constant exponent; no practical running-time claim is made.

Prior work and the point of departure. Structural restrictions on precedence yield important intermediate results. On any number of machines, Hu's highest-level-first algorithm is optimal when the precedence constraints form an in-tree [17], and Papadimitriou and Yannakakis gave a polynomial-time algorithm for interval orders [27]. Garey, Johnson, Tarjan, and Yannakakis gave a linear-time algorithm on three processors for opposing forests, which are disjoint unions of an in-forest and an out-forest [12, Section 4]. For fixed m, polynomial-time algorithms are also known for level orders and opposing forests in a setting where the number of available processors may vary over time [6], and for further special classes of precedence graphs [5]. Dolev and Warmuth obtained a running time $O(n^{h(m-1)+1})$ when the precedence graph has height h, measured in arcs, and at most m processors are available in each slot [4, Theorem 3]. The available number may vary over time. This is polynomial when both h and m are fixed. Their normalized optimal schedules, called zero-adjusted, have a slot whose at most $m-1$ jobs with successors determine the prefix and the remaining subproblem, the latter up to isomorphism. When the graph has positive height, both recursive pieces have smaller height [4, Theorem 2].

Nederlof, Swennenhuis, and Węgrzycki developed this structural approach into an exact algorithm with running time $(1+n/m)^{O(\sqrt{nm})}$ for n jobs on m machines, hence $2^{O(\sqrt{n}\log n)}$ for three machines [26, Theorem 1.1]. Their decomposition at selected slots and memoization of subproblems described by source and sink sets already control the number of subproblems despite potentially linear recursion depth [26, Sections 5–6]. Their proper-decomposition theorem assumes at most m sources [26, Definition 1 and Theorem 3.1]; their full algorithm handles arbitrary precedence graphs. The marked intervals below build on this decomposition viewpoint. We prove the required separator statements and bound the size of each global interval description directly; the polynomial bound does not follow by specializing their subexponential bound.

The classical approximation guarantees are the ratio $2-1/m$ of Graham's list scheduling [15] and the ratio $2-2/m$ of the Coffman–Graham algorithm on m machines, which is $4/3$ for three machines [19]. Approximation algorithms also revealed useful recursive structure. For fixed machine count and fixed $\varepsilon > 0$, Levey and Rothvoß obtained a $(1+\varepsilon)$-approximation using linear programming (LP) hierarchies [23]; Garg improved the running time to quasipolynomial [14]. Li replaced LP conditioning by combinatorial guesses and obtained the bound $n^{O((m^4/\varepsilon^3)(\log\log n)^3)}$ [24]. Das and Wiese later gave a simpler combinatorial quasipolynomial-time approximation scheme

[3]. These works give a related perspective on the recursive structure of nearly optimal schedules. The exact separator and global-description statements used here are established separately.

Why a finite search can suffice. After adding isolated jobs, we may assume that every slot contains three jobs. If selected slots are removed from a feasible schedule, the remaining open gaps can be scheduled independently: reordering the jobs of one gap does not change their temporal relation to any exterior job. This suggests a recursive search whose subproblems are the sets of jobs in those gaps. The obstacle is their number. Naming a gap by its two boundary triples does not by itself determine its job set, and repeatedly intersecting descriptions inherited from its ancestors can require an unbounded formula.

We prove that every feasible instance has a recursive decomposition in which each gap is described *globally*, as a subset of all jobs, by a Boolean formula of bounded size. The permitted tests are precedence to or from a triple of jobs, membership in that triple, and rank thresholds in a fixed linear extension or its reverse. The bound is independent of the depth of the decomposition. There are therefore only polynomially many sets that the algorithm must consider. Compatible selected triples connect smaller accepted gap sets, and their concatenation gives the schedule.

The structural proof chooses lexicographically minimal schedules only as existence witnesses; the algorithm enumerates the descriptions instead of computing those witnesses. Fix a priority order extending precedence, and call its current initial segment low and the remaining jobs high. A selected slot separates this cutoff when every earlier high job is a precedence predecessor of a job in that slot, and every later low job is a precedence successor of one. In a normalized interval, we promote jobs one at a time from high to low and move a separator forward. Each pair of successive selected slots separates one common cutoff: each low job in the intervening gap has a predecessor in its left slot, and each high job has a successor in its right slot.

The two sides of a center slot are treated by the same construction: for one side we reverse both time and precedence. To describe a gap without remembering all its ancestors, we maintain a short list of sets closed under taking successors in the current direction. A boundary slot limits the number of new list entries that can survive. A global boundary condition excludes distant jobs, while an exchange condition keeps that control valid after later reorderings. At a change of direction, a paired form of the inherited entries lets us replace their history by a bounded formula. The distinction between agreement inside a known interval and a description valid on all jobs is what makes this last step necessary.

Organization. Sections 2 and 3 specify a finite description family and the uniform algorithm. Section 4 supplies common cutoffs, and Section 5 uses them to form smaller child intervals. Section 6 bounds inherited predicate information; Section 7 proves the global boundary invariants. Section 8 describes whole intervals and marked splits by bounded formulas, completing the hierarchy proof. Section 9 gives the explicit running-time bound.

2 Full schedules, frames, and descriptions

We write $x \prec y$ if there is a directed path of positive length from x to y. Transitive reachability can be computed in polynomial time. All restrictions of the precedence order below mean restrictions of this transitive order, including when a path in the original graph uses vertices outside the restricted set.

2.1 Padding and interval reorderings

If $n > 3T$, reject. Otherwise add $3T - n$ isolated jobs and let J be the resulting set, with $N = |J| = 3T$. A *full schedule* of a set $W \subseteq J$ is an ordered sequence of antichain triples partitioning W, respecting $\prec$. Its length is necessarily $|W|/3$. The empty set has the empty full schedule.

Lemma 2.1. *The original deadline is feasible if and only if J has a full schedule. The padded instance has polynomial size in the stated input encoding.*

Proof. A schedule of the original jobs leaves exactly $3T - n$ vacant positions among the three machines and T slots. Fill them with the isolated jobs. Conversely, delete the isolated jobs from a full schedule. Since $T \leq n$ and the vertices are explicitly listed, $N = 3T \leq 3n$ is polynomially bounded. □

A schedule's slots will also denote their job triples. An *open interval* (a, b) consists of all slots strictly between two boundary slots; we use the same symbol W for that interval's job set. Virtual boundaries before the first and after the last slot are allowed.

Lemma 2.2. *Replacing the schedule inside an open interval by any full schedule of its job set preserves feasibility of the complete schedule.*

Proof. All exterior jobs are either before the entire interval or after it. Their order relative to every interior job is unchanged. Interior precedences hold by the assumed feasibility of the replacement. □

2.2 The two frames

Fix a linear extension of the order on J, with rank $\lambda : J \to \{1, \ldots, N\}$. A deterministic choice is obtained by successively removing a minimal job, breaking ties by its identifier. We also use the reversed order, reversed time, and reversed linear extension. These are our two *frames*. The two ranks of a job sum to $N + 1$. Unless a frame is specified otherwise, all time comparisons, precedence relations, and ranks in an argument belong to its working frame.

For an actual slot a, identified with its job triple, define global cones

$$D_a = \{x \in J : \exists y \in a,\ y \prec x\}, \qquad P_a = \{x \in J : \exists y \in a,\ x \prec y\}, \qquad \widehat{D}_a = D_a \cup a, \quad \widehat{P}_a = P_a \cup a.$$

At the left sentinel take $D_a = \widehat{D}_a = J$ and $P_a = \widehat{P}_a = \varnothing$. At the right sentinel use the opposite convention. Reversal exchanges D and P, and exchanges their weak versions. In a fixed frame the D-cones and weak D-cones are upward closed, whereas the P-cones and weak P-cones are downward closed.

Cone membership always concerns the whole poset J, even when a formula is evaluated on an interval. A job that is temporally earlier than a block need not be a predecessor of that block.

2.3 A fixed finite family

Let $\mathcal{F}$ be the family of subsets of J described by Boolean formulas with at most $K = 10000$ leaves. Leaves are the following atomic tests or their negations:

- true and false;
- rank thresholds $\lambda \leq k$ and $\lambda \geq k$, with $0 \leq k \leq N + 1$, in either frame;

- membership in a triple of jobs, or in any of its four cones $P, D, \widehat{P}, \widehat{D}$, in either frame.

Internal nodes are binary conjunctions and disjunctions. Negating a formula does not increase its number of leaves: apply De Morgan's laws and push negations to the leaves. Sentinel tests are constants. We distinguish a *global description* $W = F$, with $F \in \mathcal{F}$, from a *relative description* $L = W \cap F$.

Lemma 2.3. *The family $\mathcal{F}$ can be generated deterministically in polynomial time and represented by N-bit vectors.*

Proof. There are $O(N^3)$ atomic tests. The number of binary tree shapes and connective assignments with at most K leaves is a constant depending only on K. Labelling the leaves therefore gives $O(N^{3K})$ formulas, with a constant implicit factor. Compute their truth sets after precomputing reachability, then deduplicate by comparing the bit vectors pairwise. This is a deterministic finite procedure with polynomial cost. □

3 The algorithm and its structural certificate

The algorithm accepts or rejects each $W \in \mathcal{F}$, in increasing order of cardinality. Write $\mathsf{A}(W)$ for its acceptance value, and regard $\mathsf{A}(U)$ as false when $U \notin \mathcal{F}$. Set $\mathsf{A}(\varnothing)$ to true, and reject sets whose sizes are not divisible by three.

For each remaining nonempty W, form states

$$(L, Z, R), \qquad W = L \dot\cup Z \dot\cup R,$$

where Z is an antichain triple and $L = W \cap F$ for some $F \in \mathcal{F}$. Keep a state only if the displayed order of its three parts is *compatible*: no job in a later part precedes a job in an earlier part. This test uses the transitive order on J. Declare the state initial when $\mathsf{A}(L)$ is true and final when $\mathsf{A}(R)$ is true. Add an edge

$$(L, Z, R) \longrightarrow (L', Z', R')$$

exactly when

$$L \cup Z \subseteq L', \qquad \mathsf{A}\big(L' \setminus (L \cup Z)\big) = \text{true}. \tag{3.1}$$

All acceptance values consulted concern smaller sets. Set $\mathsf{A}(W)$ to true if an initial state reaches a final state, allowing a path consisting of a single state. Return YES precisely when $\mathsf{A}(J)$ is true.

Proposition 3.1. *This is a uniform deterministic polynomial-time algorithm. Every set it accepts has a full schedule, which the algorithm can explicitly return. With $K = 10000$, its deadline decision and schedule construction take $O((L+2)^{150020})$ deterministic multitape steps.*

Proof. For soundness, consider a path with triples $Z_1, \dots, Z_k$ and left sets $L_1, \dots, L_k$. Put

$$A_0 = L_1, \qquad A_i = L_{i+1} \setminus (L_i \cup Z_i)\ (1 \le i < k), \qquad A_k = R_k.$$

The inclusions in (3.1) give the ordered disjoint partition

$$W = A_0 \dot\cup Z_1 \dot\cup A_1 \dot\cup \cdots \dot\cup Z_k \dot\cup A_k. \tag{3.2}$$

Every A_i is smaller than W and was already accepted. By induction it has a full schedule. Every two distinct pieces in (3.2) are separated by a selected state, with the earlier piece in its left part or triple and the later piece in its triple or right part. Compatibility of that state excludes a backward

precedence between the pieces. Concatenating their schedules with the selected triples is therefore a full schedule of W.

To produce that schedule, record a predecessor whenever a state is first reached, with initial states distinguished as roots. Trace back from a reached final state and use the partition (3.2). Along every transition $|L|$ increases by at least three, so an accepting path has at most $N/3$ triples. Retrieve the already stored schedules of its smaller gap sets and concatenate them with those triples. Store this one explicit schedule for W, avoiding any repeated recursive expansion.

There are only polynomially many formula instances, sets, states, and state pairs because the leaf allowance K is fixed. Sequential scans of their bit-vector records therefore give a uniform polynomial-time implementation. Section 9 gives the explicit multitape accounting, including the scans and copying needed for reconstruction, and proves the stated exponent. □

The completeness certificate uses only intervals, their marked triples, and descriptions; it imposes no bound on the number of levels.

Definition 3.2. A *bounded-description hierarchy* for a feasible J has the whole schedule as its root interval. At a nonempty node W, one may first replace its schedule by a full schedule of the same job set. One then marks at least one actual slot; the nonempty open gaps between successive marks and the two boundaries become its children. Different children may be developed with separate copies of their witnessing exterior schedules. The following descriptions are required:

(i) every node job set belongs to $\mathcal{F}$;

(ii) for every marked triple, its earlier jobs within the node have a relative description $W \cap F$, with $F \in \mathcal{F}$, in original time.

Lemma 3.3. *If a feasible J has a bounded-description hierarchy, the algorithm accepts J.*

Proof. Induct on node size. Its marks give compatible states in chronological order by property (ii). Their end gaps and transition gaps are empty or are its children, belong to $\mathcal{F}$, and are strictly smaller because there is an actual mark. They have already been accepted by induction. The states consequently give an initial-to-final path. The root belongs to $\mathcal{F}$ because its predicate is true. □

We prove the following structural assertion in Sections 4 to 8. The considerably larger allowance $K = 10000$ avoids dependence on tight constant bookkeeping.

Theorem 3.4. *Every feasible full three-machine schedule has a bounded-description hierarchy. In fact, the global node predicates and relative split predicates needed in the hierarchy have fewer than* 500 *leaves.*

Proof of Theorem 1.1, assuming Theorem 3.4. Apply Lemmas 2.1 and 3.3, Proposition 3.1, and Theorem 3.4. The algorithm is sound on every enumerated set and complete at J. For makespan minimization, test all deadlines $1, \dots, n$ and take the smallest feasible one. A topological ordering gives a feasible schedule of length at most n, so this search succeeds. Return the stored full schedule for its padded instance and delete the isolated padding jobs. Assign the at most three remaining jobs in each slot to distinct machines. The schedule has optimum makespan by the choice of deadline. The bound in Proposition 3.1 already allows all these deadline trials. □

4 Separators with a shared cutoff

Decomposing a normalized schedule at selected slots is a central device in Dolev–Warmuth [4, Theorem 2] and in the proper-decomposition construction of Nederlof, Swennenhuis, and Węgrzycki [26, Section 3]. Here we prove the shared-cutoff form needed for the interval construction:

successive boundaries must separate the same ideal of low-priority jobs. The global-list invariants in later sections control these descriptions inside descendant intervals.

This section concerns one fixed frame and a domain consisting of a contiguous interval of a feasible schedule. A set is an *ideal* in the domain if it contains every domain predecessor of each of its members. Its complement in the domain is an upset.

Definition 4.1. Given an ideal S of low jobs in a domain, a slot z is a *separator at cutoff* S if

(a) every domain job before z that is outside S belongs to P_z;

(b) every domain job after z that belongs to S belongs to D_z.

The other domain jobs are called high. A left boundary outside the domain is also a separator when all low domain jobs descend from it; there are then no domain jobs before that boundary.

An actual separator has a useful description of its earlier jobs. For a global set $H \subseteq J$, put

$$B_z(H) = P_z \cup \big(H \setminus \widehat{D}_z\big). \tag{4.1}$$

Lemma 4.2. *If z is an actual separator for a domain U at cutoff S, and $H \cap U = S$, then $B_z(H) \cap U$ is exactly the set of domain jobs strictly earlier than z. Reversing order and time, and exchanging low with high, preserves the separator property.*

Proof. Every job in P_z is strictly earlier than z. An earlier low job is outside $\widehat{D}_z$. A low job at z belongs to $\widehat{D}_z$, and a later low job belongs to D_z by separation. Finally, every earlier high job belongs to P_z. These observations give both inclusions. Reversal exchanges the two conditions in Definition 4.1. □

4.1 Lexicographic normalization

A *priority order* is a total order of the domain extending precedence. Read a full schedule chronologically, with each triple sorted by priority. Among the finitely many feasible full schedules of the domain, choose one whose resulting sequence of priority indices is lexicographically least. Call it *normalized* for that priority. This is an existence choice, not an operation of the decision algorithm.

Lemma 4.3. *In a normalized schedule, a feasible exchange that puts a better-priority job into an earlier slot is impossible. More generally, fix a slot r. After any feasible reordering confined strictly after r, a feasible exchange of a job at r with a better-priority job at a later slot is still impossible.*

Proof. Replacing one member of a sorted triple by a better-priority job lexicographically improves that triple. In the second assertion, the schedule through r is unchanged by the intervening reordering. The proposed exchange would therefore first change the normalized sequence at r, and improve it there. The resulting full schedule would contradict minimality, regardless of what happens later. □

Lemma 4.4 (Shared-cutoff walk)**.** *Fix a normalized full schedule of a domain and let S and S^+ be consecutive priority prefixes, differing by the promotion of one job from high to low. Suppose s is a separator at cutoff S, either inside the domain or at its exterior left boundary. Then there is a separator $t \geq s$, found by advancing only to domain slots on the right, that separates both S and S^+. If the priority order is specified on a larger job set and the promoted job is outside the domain, one may take $t = s$.*

Proof. **One advance at a fixed cutoff.** Keep the normalized schedule and its priority order fixed. Let L be a prefix of that priority order, and suppose the current position s satisfies the earlier-high condition for L. If a later low job does not descend from s, choose the earliest slot $t > s$ containing such a job, and choose one of them, x. Every domain predecessor of x lies strictly before s. Indeed, that predecessor is low because L is an ideal. A predecessor at s would force $x \in D_s$. A predecessor strictly between s and t would descend from s, by the earliest choice of t, and again force $x \in D_s$.

Let y be a high domain job at a slot r with $s \leq r < t$. If y had no domain successor at or before t, exchanging x and y would be feasible: all predecessors of x are before s, and delaying y would violate no successor constraint. Moving x earlier and y later preserves their other precedence constraints. Since L is a priority prefix, x has better priority than y, contradicting Lemma 4.3. Thus y has a successor by t. The successor is high, since the high set is an upset. If it is strictly before t, repeat the argument. Strictly increasing times give a chain from y to a high job at t.

When s is internal, every earlier high job has a descendant at s by the earlier-high condition. That descendant is high, and the preceding tracing extends the chain to t. When s is the exterior left boundary, there are no earlier domain jobs. Hence t satisfies the earlier-high condition. We may advance to t and repeat until there is no later low exception. There are finitely many slots, and every advance is strict, so the final position also satisfies the later-low condition. Throughout this walk the normalized schedule itself stays fixed; only the selected separator position changes.

Adding one low job. Apply this procedure with $L = S^+$, starting at the given separator for S. The earlier-high condition for S^+ holds initially because its high set is smaller. If no advance is needed, the same position separates both cutoffs.

Otherwise the first exception is the single promoted job: every later job already low at S descends from the starting position. At the first target, the promoted job lies in that target slot, so it introduces no earlier high job for the old cutoff S. Thus the earlier-high condition holds there for both cutoffs.

In every subsequent advance, the exception is old-low: the promoted job stays at the first target, at or before the current position. An old-low exception has better priority than every old-high job. The same exchange and successor tracing therefore propagates the earlier-high condition for S as well as for S^+. At the final position, the later-low condition for S^+ also implies the later-low condition for S, since $S \subseteq S^+$. The final position separates both cutoffs.

If the promoted job is outside the domain, the two restricted prefixes coincide, so the original position already separates both. □

Corollary 4.5. *Every ideal of a nonempty set with a full schedule has an actual separator in some full schedule of that set.*

Proof. Prioritize the ideal before its complement, using a linear extension inside each. Normalize. The first actual slot separates the empty prefix. Promote the jobs of the ideal one by one and use Lemma 4.4. The final position is an actual separator for the ideal. □

The shared cutoff, rather than merely the existence of separate separators, will be used for a gap: its low jobs descend from its left endpoint, while its high jobs precede its right endpoint. Keeping the final position of each promotion is important. Intermediate positions assist the proof but need not become marks.

5 The interval construction and its invariants

We now construct the hierarchy from a feasible full schedule of J. Each node has a *tag*, one of the two frames, and a witnessing complete schedule. Descending to a child changes only the jobs inside that child; its boundary triples and exterior schedule remain fixed. Branches can retain separate witnesses, as allowed in Definition 3.2.

5.1 Global lists and boundary conditions

In the tag frame of a node $W = (a, b)$, attach an ordered list $\mathbf{K} = (K_1, \ldots, K_\ell)$ of global upsets. Define the global qualifier set and the local old-high set by

$$\mathcal{Q}(\mathbf{K}) = \bigcup_{i=1}^{\ell} K_i, \qquad O = W \cap \mathcal{Q}(\mathbf{K}).$$

For every globally qualifying job x, set

$$\mathrm{key}_{\mathbf{K}}(x) = \left(\max\{i : x \in K_i\},\ \lambda(x)\right), \tag{5.1}$$

with lexicographic comparison. Entries may overlap. Their largest index, not their smallest one, is used. Because each entry is an upset, $x \prec y$ with x qualifying implies that y qualifies and $\mathrm{key}_{\mathbf{K}}(x) < \mathrm{key}_{\mathbf{K}}(y)$.

We maintain the following three invariants in the node's tag frame.

(I1) $O \subseteq P_b$, and $W \setminus O \subseteq D_a$.

(I2) Every global qualifier scheduled at or before a belongs to $\widehat{P}_a$.

(I3) After *any* feasible reordering of W in its existing slots, there is no feasible direct exchange between $x \in W \setminus D_a$ and a qualifying job y at a when $\mathrm{key}_{\mathbf{K}}(x) < \mathrm{key}_{\mathbf{K}}(y)$.

Invariant I1 controls the jobs inside the interval; Invariant I2 controls qualifiers outside its left boundary, and Invariant I3 makes that control persist after later reorderings.

The exchange in Invariant I3 is tested for feasibility in the complete schedule. The hypothetical exchanged schedule need not preserve any decomposition or any of the list invariants. The key of x is defined because Invariant I1 implies $W \setminus D_a \subseteq O$. A sentinel has no boundary jobs, so the corresponding exchange condition is vacuous.

Lexicographic minimality within an interval controls exchanges of its interior jobs; it gives no such control over exchanges with an exterior boundary job. Invariant I3 supplies the latter control. Suppose a qualifying job $x \in W \setminus D_a$ has all its predecessors strictly before a, and its key is smaller than that of a qualifying $y \in a$. If y had no successor at or before the slot of x, their exchange would be feasible. Invariant I3 therefore forces such a successor. The proof in Section 7 uses this first successor to connect exterior qualifiers to the local separator walk.

All predicates are frozen global subsets of J once constructed. In particular, a cone used in an old predicate continues to refer to its fixed triple of jobs; it is not redefined after later reorderings. We shall prove, simultaneously with the three invariants, that each pruned list has at most five entries and each entry has fewer than 40 leaves.

At the root, use the original frame, the two sentinels, and the one-entry list (J). Invariant I1 follows from the sentinel cones, and Invariants I2 and I3 are vacuous.

5.2 A center and two open sides

In either working frame, let a, b now mean the left and right boundaries of W in that frame, and write

$$X = W \setminus D_a, \qquad Y = W \setminus P_b.$$

Invariant I1 gives

$$W \subseteq D_a \cup P_b, \qquad X \subseteq P_b, \qquad Y \subseteq D_a. \tag{5.2}$$

This cover is unchanged by reversal: the two boundary cones exchange roles. In either frame X is an ideal in W, and Y an upset. The list, O, and the other two invariants are used in their stated form only in the tag frame.

In the tag frame choose an actual center v separating the ideal $W \cap P_b$, using Corollary 4.5 and, if needed, reordering W. Mark v. At the root use the last slot as center; it separates the ideal J. The center triple and the job sets on its two sides are now fixed.

Treat each side as an open prefix $E = (a, v)$, in the frame that makes it a prefix. Call that side same if its working frame is the node's tag and switch otherwise. Recompute all boundary notation in that working frame. The center is a separator at cutoff

$$S_* = \begin{cases} W \cap P_b, & \text{same}, \\ X = W \setminus D_a, & \text{switch}. \end{cases} \tag{5.3}$$

Indeed the second formula is the complement of the original center-low set after reversing order and time, so Lemma 4.2 applies.

Use the following priority orders on W, and the following nested low cutoffs.

Same direction.
Begin with $S_0 = (W \setminus O) \cap P_b$. Order S_0 first by increasing rank, then O by increasing $\text{key}_{\mathbf{K}}$, and then Y by increasing rank. Promote the jobs of O in that key order.

Switched direction.
Begin with $S_0 = \varnothing$. Order X first and $W \cap D_a$ second, each by increasing rank in the switched frame. Promote the jobs of X in that order.

These are total orders extending precedence. For the same case, S_0 is an ideal, Y is an upset, and the key order on the middle group extends precedence. The groups partition W since $O \subseteq P_b$. For the switch case use idealness of X. Every cutoff lies in P_b on W, by (5.2), and the initial low set lies in D_a.

Normalize the open side E using the restriction of its priority order to E. This can be done independently on the two sides before constructing children. It preserves feasibility by Lemma 2.2 and preserves the center separator, since neither side's job set changes. The resulting sequence of low subsets of W is

$$S_0, S_1, \dots, S_q = S_*.$$

Some promoted jobs may lie outside E. Starting at its exterior left boundary $s_0 = a$, use Lemma 4.4 for every promotion, taking s_i to be the *final* position of that promotion. Thus

$$a = s_0 \le s_1 \le \cdots \le s_q < s_{q+1} = v. \tag{5.4}$$

The starting boundary is a separator on E because $S_0 \cap E \subseteq D_a$. For each $i < q$, both s_i, s_{i+1} separate the cutoff S_i on E. The same assertion for $i = q$ uses the center separator at v.

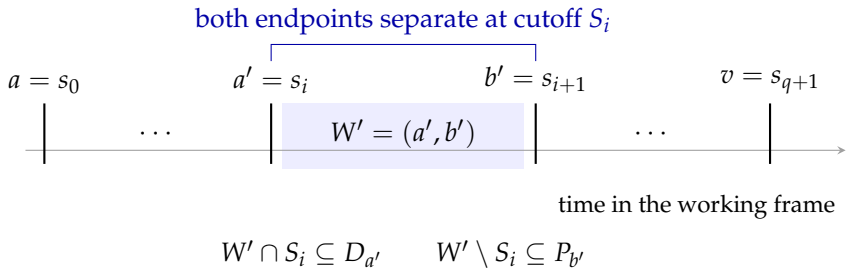


Figure 1: One side of a node, viewed in its working frame. Each nonempty gap uses the cutoff belonging to its strict walk step. The endpoints separate that same cutoff, although adjacent gaps may use different cutoffs. The horizontal arrow denotes schedule time, not a precedence relation.

Mark the distinct actual walk positions in E, in addition to v. For each strict step $s_i < s_{i+1}$, use

$$a' = s_i, \qquad b' = s_{i+1}, \qquad S = S_i \tag{5.5}$$

as its edge data; Figure 1 shows one such step. Removing repetitions from (5.4) does not create an unassigned gap: consecutive distinct positions are exactly the endpoints of one of these strict steps. If the gap $W' = (a', b')$ is nonempty, make it a child and tag it with this working frame.

Lemma 5.1. *At every child edge, b' is an actual triple in W, while a' is either a or an actual triple of E. On its child,*

$$W' \cap S \subseteq D_{a'} \cap P_b, \qquad W' \setminus S \subseteq P_{b'}. \tag{5.6}$$

Every child excludes the center and is strictly smaller than its parent. The root has no nonempty switched child.

Proof. The shared cutoff at a', b' gives the two required separator implications on the open side. All cutoffs lie in P_b on W. The remaining assertions follow from the positions of the marks, and from the root's last-slot center. □

It remains to give each child a short list selecting exactly its high jobs, and to preserve the global boundary invariants. The following sections prove these claims by a top-down induction. When a child of W is constructed, all data for W and its ancestors have already been constructed. No claim about a deeper descendant is used.

6 Short global lists and changes of direction

Fix a nonempty child $W' = (a', b')$ of $W = (a, b)$, with center v, edge cutoff S, and working frame as in (5.5). The child's tag is this working frame. At the end of an update, *prune* every entry whose intersection with W' is empty, preserving the order of the retained entries. We distinguish this pruning from the suffix operation used to form a carry.

6.1 Carry and pruning

Lemma 6.1 (Suffix carry)**.** *For a global upset list $\mathbf{K} = (K_1, \dots, K_\ell)$, fix $1 \le d \le \ell$ and an integer $0 \le r \le N + 1$, and form*

$$\mathrm{carry}_{d,r}(\mathbf{K}) = \big(K_d \cap \{\lambda \ge r\}, K_{d+1}, \dots, K_\ell\big). \tag{6.1}$$

Proof component	Interface used by the next step
Shared cutoffs	A normalized side, a starting separator, and successive priority prefixes give two boundaries separating the same cutoff (Lemma 4.4).
Child intervals	A valid parent gives a center and strictly smaller gaps with their shared cutoffs (Lemma 5.1).
Global lists	Each child receives its high-job set and first invariant through at most five upset entries, each with fewer than 40 leaves (Lemmas 6.3, 6.4 and 6.8).
Boundary stability	Parent invariants and side normalization give child Invariants I2–I3, including all future feasible child reorderings (Lemmas 7.1 and 7.2).
Finite descriptions	Valid lists and boundaries give global child sets and relative splits with fewer than 500 leaves (Lemmas 8.1 to 8.3).

Table 1: The contracts connecting the structural proof. The executable dynamic program consumes the final node sets and split sets; lists and normalized schedules are used to prove that these sets suffice.

Its qualifier set consists exactly of the old global qualifiers with old key at least (d, r). Among those jobs the new list preserves the old key order. Its entries are global upsets.

Proof. Let j be a job's largest old membership index. It is selected exactly when $j > d$, or $j = d$ and its rank is at least r. Whenever selected, its largest old membership is retained. Reindexing applies the same strictly increasing shift to these retained largest indices, and leaves the rank tiebreak unchanged. Rank suffixes are upsets because rank is a linear extension. □

Lemma 6.2 (The comparison preserved by pruning)**.** *Let a list be pruned on W'. Suppose $x \in W'$ qualifies and y is any global qualifier of the pruned list. If the pruned key of x is smaller than the pruned key of y, the same comparison held before pruning.*

Proof. Use original indices for the retained entries; their eventual reindexing preserves order. Let $M(z)$ be the largest membership index before pruning and $R(z)$ the largest retained one. Every entry containing x survives, so $R(x) = M(x)$, whereas $R(y) \le M(y)$. If $R(x) < R(y)$, then $M(x) < M(y)$. If $R(x) = R(y)$ and $\lambda(x) < \lambda(y)$, then either $M(y) > M(x)$ or the rank comparison remains the tiebreak. Both cases give the asserted old key comparison. □

6.2 The two updates

For a same-direction edge, the old list is in the working frame. If $O \setminus S = \varnothing$, its carry is empty. Otherwise let (d, r) be the smallest key of a job of $O \setminus S$, and use (6.1). Append the *injection*

$$I = D_a \cap (J \setminus P_b). \tag{6.2}$$

Then prune on W'.

For a switched edge, we shall construct a global upset G in the new working frame with the following properties:

$$W \setminus D_a \subseteq G, \qquad G \cap \{x : x \text{ is scheduled at or before } a\} = \varnothing. \tag{6.3}$$

Choose h so that the still-high part of $X = W \setminus D_a$ is exactly $X \cap \{\lambda \ge h\}$. Take $h = N + 1$ if this part is empty. Discard the old list and begin a new list with the ordered pair

$$G \cap \{\lambda \ge h\}, \qquad D_a. \tag{6.4}$$

Then prune on W'. Notice that (6.3) covers *all* of the parent's X, not only the jobs in this child.

Lemma 6.3. *For either update, assuming* (6.3) *in the switch case, the pre-pruning qualifier union on W is exactly $W \setminus S$. On these high jobs the pre-pruning key order agrees with the side's priority order. After pruning, the qualifiers on W' are exactly $W' \setminus S$, and Invariant I1 holds for the child.*

Proof. In the same case, Lemma 6.1 gives exactly $O \setminus S$ on W, in its old key order. The injection gives exactly Y: use $Y \subseteq D_a$ in (5.2). It is disjoint from the old qualifier union on W, which is contained in P_b. Since the injection is last, the pre-pruning order is the unpromoted part of O, followed by Y in rank order, as required.

In the switch case, coverage of X makes the first entry select exactly the unpromoted part of X there. The second entry selects all of $W \cap D_a$. Even if the two entries overlap, the larger index of the second puts this whole group last. The ordering therefore agrees with the switched priority.

Pruning changes no membership on W'. Its high jobs lie in $P_{b'}$, and its other jobs lie in $D_{a'}$, by Lemma 5.1. These are precisely the two assertions of Invariant I1 for the child. □

6.3 Why only five entries survive

A *run* is a chain of nodes with the same tag, starting at the root or just after a switch. Its initial entries are the root's single entry, or the two entries in (6.4). Same-direction updates only take suffixes, restrict entries by rank suffixes, and append injections. Multiple restrictions of an entry by rank suffixes in the same frame combine into one tightest threshold.

Figure 2 isolates the use of machine capacity in the following bound.

Lemma 6.4. *Every pruned list produced by this construction has at most five entries. At most three of them are injections from its current run.*

Proof. The root and switch updates have at most two initial entries. Consider a same update from W to W'. Injections retained into this child are pairwise disjoint on W. To see this, consider any pair and the time the later one was introduced. It was disjoint on its then-current parent from all previous carried entries, by Lemma 6.3. That parent contains the current W, and subsequent carries only restrict the older predicates. The disjointness persists on W.

Each surviving injection contains some $x \in W'$. It is a high job, so $x \prec z$ for some $z \in b'$. Global upward closure puts z in that same injection. The actual triple b' lies in W. Pairwise disjointness on W thus forces distinct surviving injections to contain distinct members of b'. There are only three such members. Together with the at most two initial entries this proves the bound. □

The freshly formed same-update list may have six entries before pruning. We will use the five-entry bound only for pruned lists or for a carry taken from such a list *before* appending a new injection.

6.4 Flattening the history of a run

Only the first entry of a switch pair can carry a complicated predicate from an earlier run. The following lemma removes it whenever we need to reconstruct an old cutoff on a descendant parent.

Lemma 6.5 (Local flattening)**.** *Suppose a run began with a switch out of a node Q_0, using left boundary a_0 in the resulting run frame, and entries*

$$G_0 \cap \{\lambda \geq h_0\}, \qquad D_{a_0}, \qquad Q_0 \setminus D_{a_0} \subseteq G_0.$$

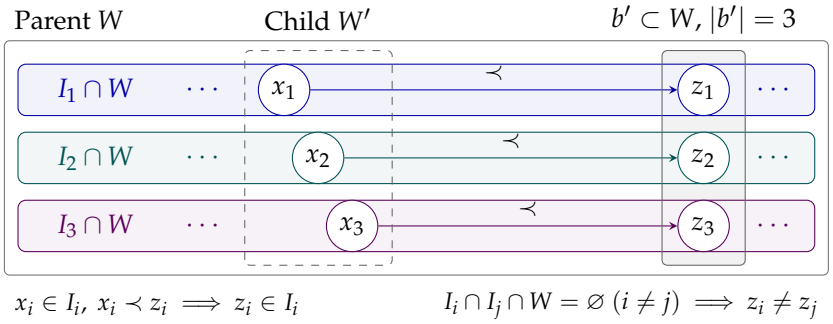


Figure 2: Why at most three injections survive a same-direction update. For each surviving injection, choose a high job x_i in the child and a successor z_i in its ending triple b'. Upward closure puts z_i in the same injection; disjointness on the parent W makes the chosen z_i distinct. The three-injection case is drawn; arrows denote precedence. Only witness pairs are shown: the injections may contain other jobs and may overlap outside W.

Let $U \subseteq Q_0$ be a later parent in that run. If the entry derived from the first displayed entry survives into an unpruned carry out of U, write its current form as $G_0 \cap \{\lambda \geq k\}$. Then the carried portion of this pair has the same union on U as the global upset

$$D_{a_0} \cup \{\lambda \geq k\}. \tag{6.5}$$

Proof. Track the original entries through later reindexings. As long as the first entry survives, a suffix operation cannot trim or discard the second entry: a cutoff at that later index would already discard the first, permanently. Thus the second entry is either still carried untrimmed, or was pruned because it was empty on some intermediate node containing U. In the latter case $D_{a_0} \cap U = \varnothing$.

On $U \cap D_{a_0}$, the second entry, when this set is nonempty, therefore selects every job. Every job of $U \setminus D_{a_0}$ lies in G_0, because $U \subseteq Q_0$. On that remainder the first entry selects exactly the rank suffix $\lambda \geq k$. This proves equality of the two unions on U. Both factors in (6.5) are upsets, so their union is globally upward closed, although it need not agree with the carried pair outside U. □

A new switch out of a node W uses the edge $U \to W$ by which W was created. Let H_- be that incoming edge's low cutoff, in its own working frame, and reserve S for the cutoff of the new outgoing edge. We need an extension of H_- that does not retain a complicated switch predicate from an earlier run. Local flattening provides it using lists and covering properties belonging to already constructed ancestors.

Lemma 6.6 (A short extension of an earlier low set)**.** *Consider an already constructed edge from a parent U to a child W, in its old working frame. Its low cutoff $H_- \subseteq U$ has a global downset extension H_-^* agreeing with it on U. This extension can be chosen with at most* 21 *leaves.*

Proof. Use the boundary notation of U in that old frame for this proof. If the edge was a switch relative to the tag of U, its low cutoff is a rank prefix of $U \setminus D_a$. Thus

$$(J \setminus D_a) \cap \{\lambda \leq r\}$$

is the required global downset, using at most two leaves.

If the edge was same, its low cutoff on U equals P_b minus its unpruned carry union, without the fresh injection. The old list on U has at most five entries by Lemma 6.4. If its run began at the root, all carry entries are simple: the root base or injections, possibly rank-trimmed. If it began at

a switch, the only possibly complicated carry entry is the first of that switch pair. If it is present in the carry, replace the carried pair by (6.5); if it is absent from the carry, every carried entry is already simple.

This gives a global upset agreeing with the carry union on U. It has at most five terms, each with at most four leaves. Indeed an injection with a rank trim has three leaves; a root base or a second switch base with a trim has at most two; and the flattened pair has two. Taking P_b minus this union gives a global downset with at most $1 + 5 \cdot 4 = 21$ leaves. An empty carry is represented by false. □

6.5 Constructing the new switch upset

Suppose a switch out of W produces a nonempty child. By Lemma 5.1, W is not the root. Let U be the parent that produced W. That preceding edge was taken in the tag frame of W, which is opposite to the current working frame. In the current frame write a_-, b_- for the boundaries of U and v_- for its center. The temporal arrangement is

$$a_- < v_- \le a < b \le b_-. \tag{6.6}$$

Here a is an actual ending endpoint of that preceding edge, viewed in reverse; it can equal v_-. The node W lies in the open suffix of U after v_-; Figure 3 shows this arrangement.

Let H_- be the low set of the preceding edge in its old working frame, regarded as a subset of all of U. The extension H_-^* from Lemma 6.6 is an upset in the current, reversed frame. Define

$$G = D_{a_-} \cap (J \setminus \widehat{P}_a) \cap (J \setminus \widehat{P}_{v_-}) \cap H_-^*. \tag{6.7}$$

Lemma 6.7. *The set G in (6.7) is a global upset satisfying both properties in (6.3).*

Proof. All four factors in (6.7) are global upsets in the current frame, so G is an upset.

Let $x \in X = W \setminus D_a$. If x had been high at the preceding edge in its old frame, Lemma 5.1 would put it in the old predecessor cone of that edge's ending endpoint. That endpoint is the current a, so reversal would give $x \in D_a$, a contradiction. Thus $x \in H_-$. Every job of H_- precedes the old right boundary of U, because all edge cutoffs lie in that cone. Equivalently, $H_- \subseteq D_{a_-}$ in the current frame. Since x is strictly after both a and v_-, feasibility excludes x from their weak predecessor cones. Finally, $x \in U$, so agreement of H_-^* on U gives $x \in H_-^*$. Hence $X \subseteq G$.

Conversely, suppose $z \in G$ is scheduled at or before a. Membership in D_{a_-}, together with (6.6), first places it strictly inside U:

$$a_- < \mathrm{time}(z) \le a < b \le b_-. \tag{6.8}$$

Only now use $H_-^* \cap U = H_-$. The old edge cutoff H_- was contained in the old center-low set. If z is at or before v_- in the current frame, the old center separator puts it in current $\widehat{P}_{v_-}$. If it is strictly after v_- but at or before a, it lies in the old normalized side and the old separator at the endpoint a puts it in current $\widehat{P}_a$. Both possibilities contradict (6.7).

These separator implications remain valid when preparing the current node: all intervening reorderings have been confined to $W = (a, b)$, and therefore have not changed any job at or before a in the current frame, the relevant boundary triples, or the fixed old cutoff sets. □

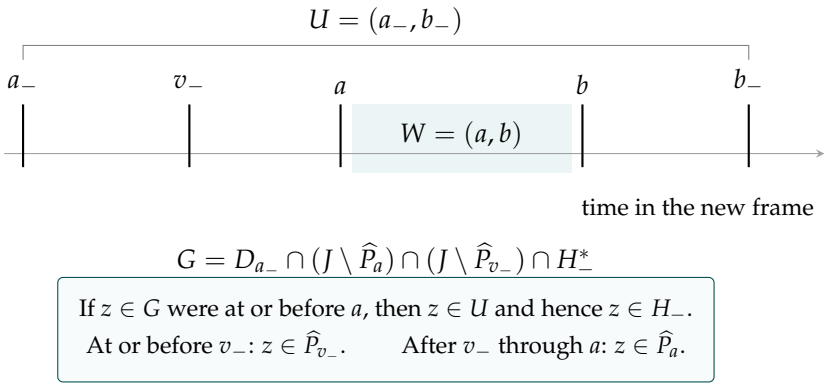


Figure 3: The ancestor configuration at a direction change, in the new working frame. A candidate early member of G is first located inside U by D_{a_-}; only there is the earlier cutoff extension used. The two weak predecessor exclusions then cover the portions at or before v_- and between v_- and a. The equality $a = v_-$ is allowed; the intervening segment then collapses. The horizontal arrow denotes time.

6.6 Predicate bounds and the induction order

Lemma 6.8. *Every entry constructed above has fewer than* 40 *leaves. This bound persists through any number of same-direction updates.*

Proof. Root bases, injections, and second switch bases are simple predicates. By Lemma 6.6, the H^*_- factor in a new G has at most 21 leaves. The three other factors of G and its outer rank suffix use four more. Thus a new first switch entry has at most 25 leaves. Further same-direction trimming only tightens that outer rank suffix. It does not append a new independent condition. The same observation applies to simple entries. □

For completeness, the construction so far has a well-founded dependency order. Assume the parent and ancestor data already satisfy their assertions. A same update is defined directly. A switch update uses the preceding edge's low set; Lemma 6.6 derives its short extension from already existing lists and, when necessary, an earlier switch's covering property. Lemma 6.7 then proves the new covering and early-exclusion properties. Lemma 6.3 supplies the high-set identity and Invariant I1; Lemma 6.4 supplies the new length bound; and Lemma 6.8 supplies the predicate bound. The length argument needs global closure and high-set identities, but not the numerical predicate bound. Thus neither bound assumes itself at a deeper node.

The root starts this induction. We next prove the remaining boundary invariants for every update, using only the parent invariants and the current normalizations.

7 Preserving the global boundary invariants

We verify Invariants I2 and I3 for a child $W' = (a', b')$ of $W = (a, b)$. All notation is in the working frame of the child edge. In a same update this is also the parent's tag frame. In a switch update the old list is not carried; the newly constructed upsets are used instead.

Preparatory changes to the schedule of W preserve the parent invariants. Its boundary triples and all earlier exterior jobs stay fixed, so Invariant I2 is unchanged. Invariant I3 already quantifies over every feasible reordering of W.

7.1 Excluding distant qualifiers

Lemma 7.1. *The updated child list satisfies Invariant I2.*

Proof. It suffices to prove the assertion before pruning, since pruning only removes qualifiers. Let z qualify for the pre-pruning child list and be scheduled at or before a'.

If $a < \mathrm{time}(z) \le a'$, the job belongs to the normalized open side E of the parent. By Lemma 6.3, it is high at the edge cutoff S. The local separator at a' puts it in $P_{a'}$ when strictly earlier, and membership in the triple puts it in $\widehat{P}_{a'}$ when its time equals a'.

Now suppose z is at or before a. The new injection $D_a \setminus P_b$ cannot contain such a job. Nor can either switch entry, by Lemma 6.7 and the strictness of D_a. Thus the only remaining case is a same-direction carry qualifier. The parent's Invariant I2 supplies a job $y \in a$ with $z = y$ or $z \prec y$. The carry is a global upset, so y also qualifies by carry. In particular a is actual. If $a' = a$, we are done.

Assume $a' > a$, and write $a' = s_i$, with edge cutoff $S = S_i$. Follow all the strict advances of the walk from a through promotion i, including intermediate advances not retained as marks. Every cutoff used in this portion of the walk is contained in S_i. On W, the final carry is exactly $O \setminus S_i$. Consequently every carry qualifier in E is high, and has worse priority than the low exceptions, at every one of these advances.

The first advance needs the parent exchange invariant, because a lies outside the normalized open side E. Consider that first actual advance off a, with earliest exception x at its target t. All *global* predecessors of x are before a. To verify this stronger readiness assertion, every predecessor is temporally before t. If one were at a, then $x \in D_a$. If one were strictly between a and t, it would belong to the contiguous side E, be low, and descend from a by earliestness of the exception. Again $x \in D_a$, a contradiction. There is no other possible predecessor at or after a.

Also $x \in X = W \setminus D_a$, so Invariant I1 puts it among the old qualifiers, outside the starting low set. It has already been promoted by stage i, so its old key is strictly below the least unpromoted key at S_i. By Lemma 6.1, the boundary job y has old key at least that least unpromoted key. The parent's Invariant I3, applied after the current preparatory reorderings of W, therefore prohibits exchanging x and y in the complete schedule.

If y had no global successor at or before t, that exchange would be feasible. All global predecessors of x lie strictly before a; moving it earlier preserves its successor constraints, and delaying y to t preserves its predecessor constraints and, under the supposition, its successor constraints. Thus y must have a strict successor by t. Such a successor is in E and is still a carry qualifier.

Any carry job strictly between a and t must likewise have a successor by t: otherwise exchange it with x within E, contradicting its lexicographic normalization. The carry is an upset, so these successors remain in the carry. Iterating along strictly increasing times reaches a carry job at t. It cannot be x, which is low and lies outside the final carry.

At every subsequent advance, the local readiness and exchange argument from Lemma 4.4 applies inside E. All carry jobs remain high at that advance, so each carry job at its starting slot has a carry descendant at its target. Tracing from the descendant of y at the first target eventually gives a descendant of y at a'. Hence $z \in \widehat{P}_{a'}$, as required. □

7.2 Exchanges after future child reorderings

Lemma 7.2. *The pruned child list satisfies Invariant I3, with its quantifier over all feasible reorderings of W'.*

Proof. Take any feasible reordering of W' in its slots. Suppose $x \in W' \setminus D_{a'}$, that $y \in a'$ qualifies for the child list, and that the new key of x is smaller than the new key of y. Invariant I1, already proved for the child in Lemma 6.3, ensures that x qualifies. By Lemma 6.2, the same comparison holds in the new list before pruning.

First let $a' > a$. Both x and y are in the parent's normalized side E, and both are high at its edge cutoff S, by Lemma 6.3. The pre-pruning key order agrees with the priority order used to normalize E. Thus x has better priority than y. The child reordering changes only slots strictly after a'. A feasible exchange of x, y in the complete schedule would therefore give a feasible full schedule of the same E with a better prefix at a', contrary to Lemma 4.3.

Now let $a' = a$. In the switch case there is no new qualifier at a, so no such y exists. In the same case neither x nor y belongs to the new injection: $x \notin D_a$, and no job at a is a strict descendant of a. Their pre-pruning comparison therefore uses only carry entries. By Lemma 6.1, it implies the same old key comparison in the parent's list. Moreover $x \in W \setminus D_a$. The preparatory reorderings and the arbitrary subsequent reordering of W' together form a feasible reordering confined to W. The parent's Invariant I3 rules out the proposed exchange. □

Proposition 7.3. *Starting from any feasible full schedule, the marking and list construction is defined at every node. Its pruned lists are global upset lists of length at most five, with fewer than* 40 *leaves per entry, and satisfy Invariants I1–I3. Every branch terminates.*

Proof. Proceed by depth from the root data of Section 5. Given valid ancestor and parent data, the center and the two normalized sides exist by Corollary 4.5 and Lemma 2.2. The shared walk and Lemma 5.1 define the children. For a same update the new list is direct. For a switch, Lemmas 6.6 and 6.7 construct the needed global upset using only earlier data, as explained at the end of Section 6. Then Lemmas 6.3, 6.4 and 6.8 give the first invariant and the bounds. Lemmas 7.1 and 7.2 give the other two invariants without using a claim about any deeper child. This completes the simultaneous induction. Every child omits the center, so its size decreases strictly. □

Invariant I2 will let a bounded predicate distinguish the child from qualifiers arbitrarily far in the past. Invariant I3 is what allows that first invariant to continue propagating after recursive reorderings. Neither is replaced by an assertion only about local separators.

8 Global interval predicates and completion

We now describe each child as a global set without intersecting with a recursively described parent. This is the step that converts the construction into a polynomial family of subproblems.

8.1 Global tests for local cutoffs

The cutoff predicates in this section are used only in the final interval and split formulas. The list construction and its entry-size bound are now complete, so these formulas may retain the existing bounded entries. The historical extension in Lemma 6.6, by contrast, had to flatten an earlier run before a new switch entry could be constructed.

For every walk cutoff S in a parent W, choose a global predicate $\widetilde{S}$ satisfying

$$\widetilde{S} \cap W = S. \tag{8.1}$$

In the same case, let $\mathbf{K}_{\text{carry}}$ be the carry determined by $O \setminus S$, before the new injection. Use

$$\widetilde{S} = P_b \setminus \mathcal{Q}(\mathbf{K}_{\text{carry}}). \tag{8.2}$$

On W, the carry selects precisely $O \setminus S$, while the low jobs are the initial $(W \setminus O) \cap P_b$ and the promoted portion of O. Thus (8.1) holds. In the switch case use

$$\widetilde{S} = (J \setminus D_a) \cap \{\lambda \le r\}, \tag{8.3}$$

where r is the rank of the last promoted member of X, or $r = 0$ before any promotion. These formulas are available at every walk cutoff, whether or not the associated open gap is nonempty. They are not required to have any prescribed meaning outside W.

8.2 A child as a global set

Figure 4 summarizes the localization steps in the following formulas.

Lemma 8.1. *In the valid construction of Proposition 7.3, let $W' = (a', b')$ be a child of $W = (a, b)$, with parent center v, edge cutoff S, new pruned list $\mathbf{K}'$, and cutoff predicate $\widetilde{S}$ satisfying (8.1). In the edge's working frame, its high and low parts are the following global sets:*

$$W' \setminus S = \mathcal{Q}(\mathbf{K}') \cap P_{b'} \cap (J \setminus \widehat{P}_{a'}), \tag{8.4}$$

$$W' \cap S = D_{a'} \cap P_b \cap (J \setminus \widehat{D}_{b'}) \cap (J \setminus \widehat{D}_v) \cap \widetilde{S}. \tag{8.5}$$

Proof. Every high child job qualifies, precedes b', and, because it is later than a', is outside $\widehat{P}_{a'}$. Conversely, a job selected by the right side of (8.4) is strictly before b' by precedence. If it were at or before a', the child's global Invariant I2 would put it in $\widehat{P}_{a'}$. It is therefore strictly in W', and the child list selects exactly its high jobs.

For (8.5), consider first a job selected by its right side. The two positive tests $D_{a'} \cap P_b$ locate it strictly inside the parent W: its time is after $a' \ge a$ and before b. This remains true with sentinel boundaries under our conventions. We can consequently use (8.1) to infer that the job is in S. This cutoff is contained in the center-low set S_*. A selected job at or after v would therefore lie in $\widehat{D}_v$, by the center separator or by membership in v itself. The corresponding exclusion places it in the open side $E = (a, v)$. Within E, a low job at or after b' is in $\widehat{D}_{b'}$, using the local separator at b'. The remaining exclusion puts the selected job strictly between a' and b'.

Conversely, a low job of this gap lies in $D_{a'} \cap P_b$ by Lemma 5.1, and it satisfies $\widetilde{S}$ by (8.1). Its time is strictly before both b' and v, so it is in neither of their weak successor cones. It passes every test in (8.5). □

The localization order matters. Neither the cutoff extension $\widetilde{S}$ nor the flattened historical extension needs to agree with its intended local set everywhere in J. Positive cone tests first place the candidate in the domain where agreement is known. The construction never appends a membership predicate for all ancestor intervals.

8.3 Relative predicates for the marked splits

Lemma 8.2. *Every marked slot z in a node W has a bounded relative description of its earlier jobs in either the working direction that produced it or the original direction. The formulas use the stated atomic language and have fewer than* 500 *leaves, as counted in Lemma 8.3.*

Proof. Use the working frame for a side, and put

$$H_* = \begin{cases} P_b, & \text{same}, \\ J \setminus D_a, & \text{switch}. \end{cases}$$

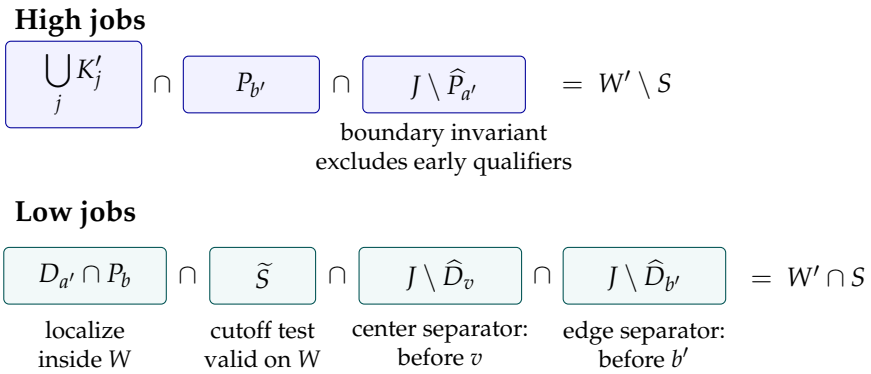


Figure 4: The two global tests for a child. For the low part, positive cones locate a candidate inside the parent before its cutoff predicate is used; the two weak successor exclusions then restrict that candidate to the gap. For the high part, the global past-boundary invariant excludes qualifiers at or before a'. The intersection factors are arranged in the order used by the proof.

For the center v, the predicate $B_v(H_*)$, restricted to W, is its earlier set, by Lemma 4.2 and (5.3).

For an internal mark z in the open side E, choose a walk cutoff S for which it is a local separator. The predicate

$$F_z = B_v(H_*) \cap B_z(\widetilde{S}) \tag{8.6}$$

gives its earlier set upon restriction to W. The first factor selects E on W; on that domain the second is the separator description from Lemma 4.2.

If the working frame is opposite to original time, $F_z \cap W$ describes the original *later* jobs. Replace F_z by

$$(J \setminus F_z) \setminus z \tag{8.7}$$

to obtain the original earlier jobs after restriction to W. The exclusion of the marked triple is necessary: complementation alone includes it. Apply the same conversion to the center predicate when appropriate. □

Lemma 8.3. *The predicates in Lemmas 8.1 and 8.2 have fewer than* 500 *leaves.*

Proof. A pruned list or a carry contains at most five entries with fewer than 40 leaves each, so its union has at most 195 leaves. An empty union uses the false constant. The same-direction cutoff (8.2) therefore needs at most 196 leaves; the switched one needs two. Use 205 as a common loose allowance.

The high predicate (8.4) uses at most $195 + 2 = 197$ leaves. The low predicate (8.5) uses at most $205 + 4 = 209$. Their union is a global description of W' with at most 406 leaves.

The center predicate has three leaves. The internal split (8.6) has at most $3 + (205 + 2) = 210$, and conversion (8.7) adds one. Complementation does not increase leaf counts. Every cone uses an actual triple or a sentinel constant; all ranks and thresholds are from the two allowed frames. Thus these are formulas in the stated atomic language. □

Proof of Theorem 3.4. Use Proposition 7.3 to construct marks and lists from a feasible full schedule. The root set J is represented by true. Every nonempty child has the global description supplied by Lemma 8.1. Every mark has the relative split description supplied by Lemma 8.2. Lemma 8.3 places both kinds of description in $\mathcal{F}$, independently of their depth. All branches terminate because the node sizes decrease strictly. These are exactly the conditions of Definition 3.2. □

9 Running time in the explicit input model

The preceding structural proof establishes completeness of the finite search. We now give the detailed deterministic multitape accounting promised in Proposition 3.1. Its purpose is to make the polynomial-time assertion uniform; the structural argument does not require efficient normalization of a schedule or a small recursion depth.

Running-time bound in Proposition 3.1. First specify a conservative implementation using sequential scans. Write $B = C_K N^{3K}$ for an upper bound on the number of enumerated formulas, where C_K depends only on the fixed K. There are at most B distinct sets and at most $S = BN^3$ candidate states for each W. Store their bit vectors, reachability marks, and predecessor record numbers on work tapes. Each state record uses $O_K(N)$ bits: its indices require only $O_K(\log N)$ bits. Store an explicit full schedule with each previously accepted set. This table uses $O_K(BN^2)$ bits, since a schedule lists at most N job identifiers and their slots.

A complete scan to fetch or update a state record costs $O_K(SN) = O_K(BN^4)$ steps. A lookup in the acceptance table by comparing all bit vectors costs $O_K(BN^2)$ steps. Even checking all required precedence relations by repeated scans of the N-by-N reachability table costs at most $O(N^4)$. Thus one ordered state-pair test and reachability update costs $O_K(BN^4)$ steps, without constant-time random access. Starting from the initial states, make at most S passes over all ordered pairs, marking a target when its source is marked and (3.1) holds. These passes find every reachable state. For all sets together their cost is at most

$$O_K\big(BS^3BN^4\big) = O_K(B^5N^{13}) = O_K(N^{15K+13}).$$

Generating, evaluating, and deduplicating formulas by scans costs at most $O_K(B^2N^4)$; reachability preprocessing, state construction, and table initialization also fit within the displayed bound. The subscript K denotes a constant fixed independently of the input. Since $N \le 3n$ and the vertices are explicitly listed, the looser bound $O((L+2)^{150020})$ covers input processing and even all n deadline trials needed for optimization. □

Remark 9.1. For the deadline feasibility decision, the same preprocessing can handle an encoding that specifies an enormous number of isolated vertices only through a binary value of n. After checking $n \le 3T$, let d be the number of vertices appearing in edges. If $T \ge d$, accept: schedule those d vertices sequentially in topological order and fill free positions with the isolated vertices. Otherwise $N = 3T < 3d \le 6|E|$, so expansion is polynomial. This extra branch is not needed for the explicit encoding in Theorem 1.1.

10 Conclusion

The finite search succeeds because every interval in a suitable recursive schedule decomposition has a global description of bounded size, independent of its depth. Local separator descriptions alone do not give that guarantee: the short upset lists, the exterior boundary invariants, and the simplification at direction changes prevent the accumulation of ancestor information.

The resulting algorithm constructs an exact optimum for the stated three-machine model. The fixed leaf allowance 10000 and the explicit large exponent establish polynomial time for the stated model. The proof supplies no practical performance guarantee.

References

[1] Hans L. Bodlaender and Michael R. Fellows. W[2]-hardness of precedence constrained K-processor scheduling. *Operations Research Letters*, 18(2):93–97, 1995. doi: 10.1016/0167-6377(95)00031-9.

[2] Edward G. Coffman, Jr. and Ronald L. Graham. Optimal scheduling for two-processor systems. *Acta Informatica*, 1(3):200–213, 1972. doi: 10.1007/BF00288685.

[3] Syamantak Das and Andreas Wiese. A simpler QPTAS for scheduling jobs with precedence constraints. In Shiri Chechik, Gonzalo Navarro, Eva Rotenberg, and Grzegorz Herman, editors, *30th Annual European Symposium on Algorithms (ESA 2022)*, volume 244 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 40:1–40:11. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022. doi: 10.4230/LIPIcs.ESA.2022.40. URL https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ESA.2022.40.

[4] Danny Dolev and Manfred K. Warmuth. Scheduling precedence graphs of bounded height. *Journal of Algorithms*, 5(1):48–59, 1984. doi: 10.1016/0196-6774(84)90039-7.

[5] Danny Dolev and Manfred K. Warmuth. Scheduling flat graphs. *SIAM Journal on Computing*, 14(3):638–657, 1985. doi: 10.1137/0214047.

[6] Danny Dolev and Manfred K. Warmuth. Profile scheduling of opposing forests and level orders. *SIAM Journal on Algebraic and Discrete Methods*, 6(4):665–687, 1985. doi: 10.1137/0606066.

[7] M. Fujii, T. Kasami, and K. Ninomiya. Optimal sequencing of two equivalent processors. *SIAM Journal on Applied Mathematics*, 17(4):784–789, 1969. doi: 10.1137/0117070.

[8] M. Fujii, T. Kasami, and K. Ninomiya. Erratum: Optimal Sequencing of Two Equivalent Processors. *SIAM Journal on Applied Mathematics*, 20(1):141, 1971. doi: 10.1137/0120018. URL https://epubs.siam.org/doi/epdf/10.1137/0120018.

[9] Harold N. Gabow. An almost-linear algorithm for two-processor scheduling. *Journal of the ACM*, 29(3):766–780, 1982.

[10] M. R. Garey and D. S. Johnson. Scheduling tasks with nonuniform deadlines on two processors. *Journal of the ACM*, 23(3):461–467, 1976.

[11] M. R. Garey and D. S. Johnson. Two-processor scheduling with start-times and deadlines. *SIAM Journal on Computing*, 6(3):416–426, 1977. doi: 10.1137/0206029.

[12] M. R. Garey, D. S. Johnson, R. E. Tarjan, and M. Yannakakis. Scheduling opposing forests. *SIAM Journal on Algebraic and Discrete Methods*, 4(1):72–93, 1983. doi: 10.1137/0604011.

[13] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[14] Shashwat Garg. Quasi-PTAS for scheduling with precedences using LP hierarchies. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, volume 107 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 59:1–59:13. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018. doi: 10.4230/LIPIcs.ICALP.2018.59. URL https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICALP.2018.59.

[15] R. L. Graham. Bounds for certain multiprocessing anomalies. *Bell System Technical Journal*, 45 (9):1563–1581, 1966. doi: 10.1002/j.1538-7305.1966.tb01709.x.

[16] R. L. Graham, E. L. Lawler, J. K. Lenstra, and A. H. G. Rinnooy Kan. Optimization and approximation in deterministic sequencing and scheduling: a survey. *Annals of Discrete Mathematics*, 5:287–326, 1979. doi: 10.1016/S0167-5060(08)70356-X. URL https://ir.cwi.nl/pub/18052.

[17] T. C. Hu. Parallel sequencing and assembly line problems. *Operations Research*, 9(6):841–848, 1961. doi: 10.1287/opre.9.6.841.

[18] David S. Johnson. The NP-completeness column. *ACM Transactions on Algorithms*, 1(1): 160–176, 2005. doi: 10.1145/1077464.1077476.

[19] Shui Lam and Ravi Sethi. Worst case analysis of two scheduling algorithms. *SIAM Journal on Computing*, 6(3):518–536, 1977. doi: 10.1137/0206037.

[20] E. L. Lawler, J. K. Lenstra, A. H. G. Rinnooy Kan, and D. B. Shmoys. Sequencing and scheduling: Algorithms and complexity. In S. C. Graves, A. H. G. Rinnooy Kan, and P. H. Zipkin, editors, *Logistics of Production and Inventory*, volume 4 of *Handbooks in Operations Research and Management Science*, chapter 9, pages 445–522. North-Holland, 1993. doi: 10.1016/S0927-0507(05)80189-6.

[21] J. K. Lenstra and A. H. G. Rinnooy Kan. Complexity of scheduling under precedence constraints. *Operations Research*, 26(1):22–35, 1978. doi: 10.1287/opre.26.1.22.

[22] J. K. Lenstra and A. H. G. Rinnooy Kan. Two open problems in precedence constrained scheduling. In M. Pouzet and D. Richard, editors, *Orders: Description and Roles*, volume 23 of *Annals of Discrete Mathematics*, pages 509–522. North-Holland, 1984. URL https://ir.cwi.nl/pub/1850.

[23] Elaine Levey and Thomas Rothvoß. A $(1+\epsilon)$-approximation for makespan scheduling with precedence constraints using LP hierarchies. In *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing*, STOC '16, pages 168–177. Association for Computing Machinery, 2016. doi: 10.1145/2897518.2897532. URL https://doi.org/10.1145/2897518.2897532.

[24] Shi Li. Towards PTAS for precedence constrained scheduling via combinatorial algorithms. In Dániel Marx, editor, *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 2991–3010. Society for Industrial and Applied Mathematics, 2021. doi: 10.1137/1.9781611976465.178. URL https://doi.org/10.1137/1.9781611976465.178.

[25] Matthias Mnich and René van Bevern. Parameterized complexity of machine scheduling: 15 open problems. *Computers & Operations Research*, 100:254–261, 2018. doi: 10.1016/j.cor.2018.07.020. URL https://arxiv.org/abs/1709.01670.

[26] Jesper Nederlof, Céline M. F. Swennenhuis, and Karol Węgrzycki. A subexponential time algorithm for makespan scheduling of unit jobs with precedence constraints. In *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 535–552. Society for Industrial and Applied Mathematics, 2025. doi: 10.1137/1.9781611978322.16.

[27] Christos H. Papadimitriou and Mihalis Yannakakis. Scheduling interval-ordered tasks. *SIAM Journal on Computing*, 8(3):405–409, 1979. doi: 10.1137/0208031.

[28] P. Schuurman and G. J. Woeginger. Polynomial time approximation algorithms for machine scheduling: Ten open problems. *Journal of Scheduling*, 2(5):203–213, 1999. doi: 10.1002/(SICI)1099-1425(199909/10)2:5<203::AID-JOS26>3.0.CO;2-5.

[29] Jeffrey D. Ullman. NP-complete scheduling problems. *Journal of Computer and System Sciences*, 10(3):384–393, 1975. doi: 10.1016/S0022-0000(75)80008-0.